

Real-time Optimal Power Allocation for Smart Grid System via Deep Neural Network: A Learning Based Approach

Fanghong Guo¹, Bowen Xu¹, Wen-An Zhang¹, Dan Zhang¹, Li Yu¹

1. Department of Automation, Zhejiang University of Technology, Hangzhou 310032, China,
E-mail: fhguo@zjut.edu.cn, wazhang@zjut.edu.cn, dzhang@zjut.edu.cn, lyu@zjut.edu.cn

Abstract: This paper attempts to solve the optimal power allocation (OPA) problem for smart grid system in a new distinguished way. Conventionally the numerical optimization approaches including traditional convex optimization and heuristic search methods almost occupy the addressing of such problem. However, these optimization algorithms may suffer from high computational complexity when the system scales up, which would inevitably create a gap between the theoretical algorithm design and real-time algorithm implementation. In this paper, we aim to provide a new learning based approach to handle the real-time OPA problem in smart grid system. The key idea behind this approach is to treat the input and output of traditional OPA optimization algorithm as an unknown nonlinear mapping, which is then approximated by recent popular learning based tools such as deep neural network (DNN). As long as the constructed DNN can accurately learn such nonlinear mapping, then the OPA problem can be solved in real time. Our main contribution is to theoretically show that the traditional decentralized gradient-based optimization algorithm for OPA problem can be accurately approximated by a well-constructed DNN. Furthermore, experimental case studies validate the effectiveness and advantages of our proposed method.

Key Words: Learning to optimize, deep neural networks, optimal power allocation, gradient-descent algorithm, power system.

1 Introduction

The current power grid is undergoing a thorough evolution towards a more efficient, economical, and environmentally sustainable power system. The concept of smart grid (SG) is proposed recently by employing advanced monitoring, control, and communication technologies, which aims to deliver reliable energy supply, enhance operational efficiency and provide flexibility for consumers [1]. It has been revealed that the success of SGs requires tighter integration of global optimization and local control. The global optimization problems usually involve optimal power allocation whose objective is to minimize the overall power generation cost or reduce the transmission line power loss while meeting the supply and demand constraint as well as local generation constraints. For decades, several optimization algorithms have been proposed to solve such kind of problem, including convex optimization approaches and some heuristic search methods (see [2] - [9]).

Although these algorithms have shown several excellent performances on both theoretical analysis and numerical simulation validation, we still face many challenges to implement them in practice. Two key challenging issues are computational complexity and real-time processing. It is because most existing optimization algorithms (including traditional convex optimization algorithms and heuristic search-based approaches) are designed in an iterative way to find the optimal solution, which is usually time-

consuming. Notably, for gradient-based convex optimization approaches, as the diminishing step size is used, it usually takes up to ten thousand steps to converge to the solution [5]. On the other hand, to converge in fewer steps, the algorithms are usually designed in a quite complicated way that would inevitably bring higher computational cost. For example, in [7], an ADMM-based optimization algorithm is utilized to solve the OPA problem in smart grid. As in each iteration, the solver needs to handle an individual constrained optimization problem, and the overall computation burden is not that cheap. Also, it is worthy to point out that if the total load demand changes, the entire optimization algorithm needs to be re-conducted. These features make the real-time implementation of existing algorithms quite challenging, especially in the modern smart grid system, where large amounts of renewable energy generators are penetrating the existing distribution power system. The fast fluctuating renewable energy sources require a real-time OPA algorithm, which unfortunately can not be easily achieved by existing optimization approaches.

Let us re-consider the OPA problem differently. It is a problem trying to seek the optimal solution for given total load demand. From the system point of view, the existing OPA optimization algorithm is indeed a system with the input being the total load demand P_d and the outputs being the optimal power generation for each distributed generator (DG), i.e., $P_i^*, \forall i$, as illustrated in Fig. 1(a). All the existing algorithms are just doing one thing, which is trying to build rigorous mathematical derivation to obtain the output P_i^* from the input P_d . Motivated by recent progress in ma-

The work was supported in part by National Natural Science Foundation of China under Grant 61903333, in part by Zhejiang Provincial Natural Science Foundation of China under Grant LQ19F030008.

chine learning technologies [10]- [13], it is natural to ask the following question: *Can we treat the existing OPA optimization algorithm as a “black box” and try to learn the relation between its input and output?* The answer should be definite. Several pioneering works are employing artificial neural networks to solve the economic dispatch problem in power system, which can be dated back to the 1990s, e.g., [14], [15]. However, due to the limited computational source at that time, only one hidden layer is considered in their network. What’s more, these works do not explicitly point out why they can approximate such complicated algorithms.

In this paper, we attempt to employ a deep neural network (DNN) to learn the relationship between input P_d and outputs P_i^* of one existing gradient-based optimization algorithm proposed in [8]. Our intuitive idea is that if such DNN can well approximate an OPA algorithm, e.g., the approach in [8], then it would be quite computationally cheap by merely passing the given load demand P_d to produce the optimal power output P_i^* , as shown in Fig. 1(b). Compared to conventional optimization algorithms, the proposed learning-based approach only requires simple matrix-vector multiplication, which could significantly benefit us in computational time saving and thus is quite suitable for real-time optimal power allocation. Besides, it is worthy of pointing out that training such a network is relatively cheap since the training samples can be easily obtained by running the optimization algorithm to be learned. In summary, the main contribution of this paper can be summarized as follows. 1) A novel deep learning-based optimization approach is proposed to solve the real-time OPA problem; 2) It is proved in theory that how many layers and neurons are needed for a deep neural network to well approximate a given optimization algorithm in [8].

2 Optimal Power Allocation in Smart Grid

Microgrid (MG), as one of the promising and representative blocks in smart grid systems, is considered in this paper. An MG could operate in both grid-connected mode and islanded mode. In the grid-connected mode, it can be regulated directly as a virtual power plant to import/export power to the main grid. While in islanded mode, it can also be controlled to share power economically [2].

2.1 Problem formulation

Optimal power allocation in MGs aims to allocate the power output among dispatchable generators at lowest generation cost subject to the total load demand constraint and local generation constraints. It is usually formulated as an optimization problem as follows.

$$\begin{cases} \min_P \sum_{i \in DG} f_i(P_i) \\ s.t. \sum_{i \in DG} P_i = P_d \\ P_i^{\min} \leq P_i \leq P_i^{\max}, \quad \forall i \in DG \end{cases} \quad (1)$$

where f_i is the generation cost function for the i th DG, which is usually approximated by the following quadratic function

$$f_i(P_i) = \alpha_i P_i^2 + \beta_i P_i + \gamma_i \quad (2)$$

with α_i , β_i and γ_i being the positive coefficients of the cost function, $P = [P_1 \ \cdots \ P_n]^\top$ denotes the aggregated vector of power output from all the dispatchable generators, with $n = |DG|$ being the numbers of dispatchable generators, P_d is the total load demand in an MG system, $P_i^{\min}$ and $P_i^{\max}$ denote the lower and upper bound of the local generation respectively.

2.2 Hierarchical decentralized optimization algorithm in [8]

It is noted that the problem formulated in (1) is a convex optimization problem. Various optimization algorithms are capable of addressing the issue such as conventional convex optimization techniques including quadratic programming, Lagrangian relaxation, and heuristic techniques such as particle swarm and genetic algorithms [7]. Recently, to decrease the computational algorithm cost and increase the design flexibility, distributed optimization algorithms have been widely employed in solving the OPA problem in smart grid. Its main idea is to decompose the conventional single computational node into several distributed nodes. Each DG has its computational node and can also exchange information with its immediate neighboring DGs to achieve the optimal global solution.

In this paper, we first adopt a hierarchical decentralized optimization (HDO) algorithm proposed in [8] to solve the OPA problem formulated in (1). For convenience, its pseudo algorithm code is summarized below.

Algorithm 1 Pseudo code of HDO algorithm in [8]

```

Initialize the outputs  $P_i(0) \in \mathbb{R}$ 
Set  $l = 0$ 
repeat
    Gradient-descent operation
         $W_i(l) = P_i(l) - \zeta_l \nabla f_i(P_i(l))$ 
    Local constraint projection
         $S_i(l) = P_{X_i}[W_i(l)] = [W_i(l)]_{P_i^{\min}}^{P_i^{\max}}$ 
    Global constraint projection
         $X_i(l) = P_{X_g}[P_i(l)] = P_i(l) - \frac{\sum_{i=1}^n P_i(l) - P_d}{n}$ 
    Coordinate operation
         $P_i(l) = \frac{S_i(l) + (n-1)P_i(l) + X_i(l)}{n+1}$ 
    Iteration update
         $l = l + 1$ 
until Some stopping criteria is met
Output  $P_i(l+1)$ 

```

Noted that similar to most exiting optimization algorithms, the HDO algorithm tries to obtain the optimal solution P_i^* in an iterative fashion. In one iteration, the gradient-descent operation, local constraint projection, global constraint projection, and coordinate operation are conducted, whose computational cost is, in fact, not so cheap. Besides, it is worth pointing out that the diminishing stepsize ζ_l is used in HDO algorithm, which results in a quite slow convergence rate. These features make it very challenging to implement the HDO algorithm in real-time optimal power allocation for smart grid. In the next section, we aim to re-solve the OPA problem in a learning-based way.

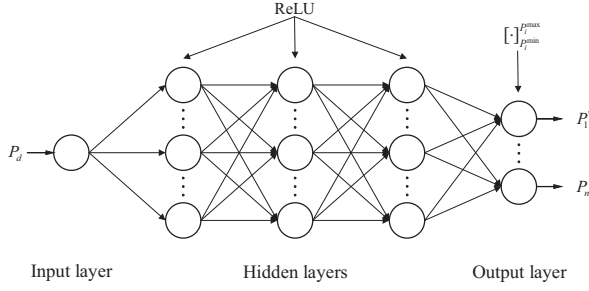


Figure 1: The DNN structure used in this paper

3 A Deep Neural Network Based Approach

In this section, we present a new DNN-based approach to solve the OPA problem in smart grid. At first, the detailed algorithm design and implementation procedure is provided, which followed by a specialized theoretical analysis of DNN approximation for HDO algorithm in [8].

3.1 Algorithm design

The algorithm design details including network structure, training stage and testing stage are introduced as follows.

3.1.1 Network Structure

Similar to the recent work [10], a fully connected DNN with one input layer, multiple hidden layers, and one output layer is utilized in this paper, whose structure is shown in Fig. 2. The input of this network is the total load demand P_d , while the output of the network is the optimal power allocation for each DG, i.e., P_i^* . In addition, we also employ ReLU function ($\max(\cdot, 0)$) as the activation function in the hidden layers. Meanwhile, in the output layer, a projection operator is used as a special activation function to meet the local power output constraint, whose implementation is the same as that shown in Fig. 3.

3.1.2 Training Stage

In the training stage, the training examples are generated by running the HDO algorithm under different load demand conditions P_d . The obtained optimal power output $P_i^{*(k)}$, $i \in DG$ together with the corresponding load demand $P_d^{(k)}$ constitute the k th training sample, where the superscript (k) denotes the index of the training samples. Repeating running the HDO algorithm with different load demand $P_d^{(k)}$, we can finally get the entire training data set $(P_d^{(k)}, \{P_i^{*(k)}\})$. In addition, extra validation data sets are also generated to perform the cross-validation. Note that compared to the training data set, the size of the validation data set is much smaller.

Once the training and validation data sets are ready, we firstly use the training data set to optimize the weights of the DNN so that it gradually approaches our desired output. The cost function we use is the mean square error (MSE)

between label $\{P_i^{*(k)}\}$ and the output of the DNN. The optimization algorithm for training used here is *Adam*, which is most commonly used in deep learning recently [20]. This optimization algorithm is updated from the RMSprop optimizer and AdaGrad optimizer. The first-moment estimation and second-moment estimation of the gradient are used to limit the learning rate of each training step to a clear range. Such a range can also be adjusted automatically. We select the proper learning rate and batch size by cross-validation. To further improve the training performance, the weights are initialized by using the truncated normal distribution.

3.1.3 Testing Stage

In the testing stage, we first feed different load demand P_d into the well-trained DNN and collect its outputs, i.e., P_i^* . Note that the feeding load demand P_d may be different from that in the training stage. Then we compare the results obtained by the traditional HDO algorithm and proposed DNN-based method to evaluate the effectiveness.

3.2 Theoretical analysis in DNN approximation

It has been revealed in many existing works (e.g., [16], [10], [17], [18], [19]) that DNN can be utilized to approximate certain kinds of nonlinear functions and some “learnable” iterative algorithms. However, it remains unclear whether the proposed DNN can well approximate the existing OPA algorithms. In addition, it is also unclear that the well-approximated DNN is a finite-size network or not. In other words, at most how many hidden layers and neurons are needed for such DNN to approximate the OPA algorithm within given approximation accuracy. In this subsection, based on the HDO algorithm in [8], we try to theoretically analyze the DNN approximation accuracy by giving a quantitative number of the hidden layers and its neurons, summarized in the following Theorem.

Theorem 1 Consider a well-trained deep neural network with the structure presented in Section 3.1.1, and the training strategy illustrated in Section 3.1.2. It can well approximate Algorithm 1 (HDO algorithm) with no errors by employing $O(l) = 6l$ hidden layers and $O(nl) = 4nl$ neurons, where l is the iteration step needed for HDO algorithm, and n is the total number of dispatchable generators.

Proof. The main steps of the DNN approximation analysis are outlined as follows:

- Step 1: Construct simple neural networks that may consist of ReLUs to represent the mapping between adjacent variables in one iteration;
- Step 2: Compose these small neural networks to approximate a rational function representing one iteration of the algorithm;
- Step 3: Concatenate these rational functions to approximate the entire algorithm;
- Step 4: Analyze the error between the above network and traditional numerical calculation methods.

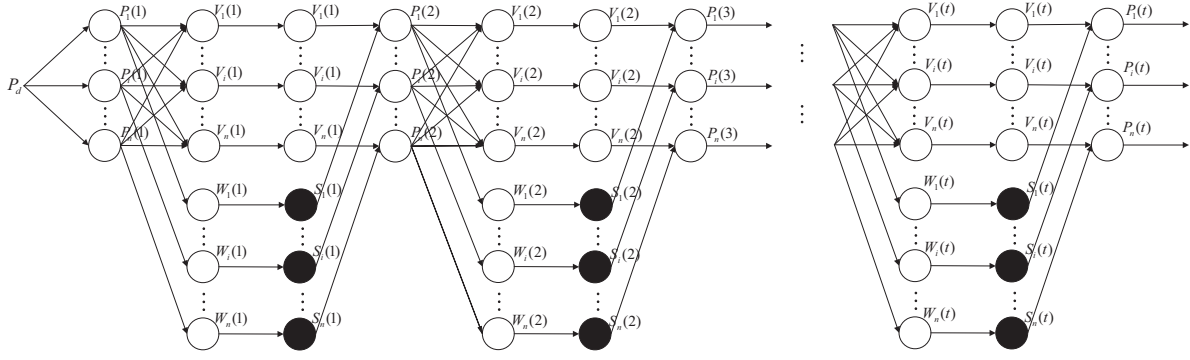


Figure 2: Implementation of HDO algorithm in [8]

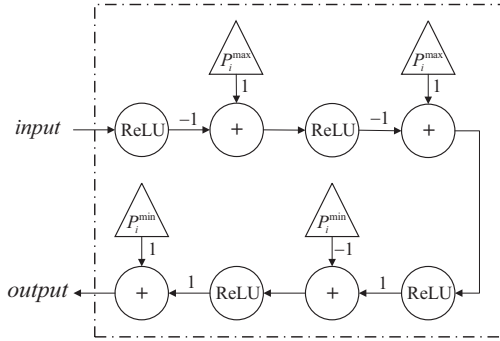


Figure 3: Implementation of the projection operator

It is well-known that a combination of linear transformations and nonlinear activation functions such as ReLU can be implemented by DNN. For Algorithm 1, we first focus on the transition between adjacent variables. Obviously, the gradient of the cost function can be written as $\nabla f_i(P_i(l)) = 2\alpha_i P_i(l) + \beta_i$ which means $W_i(l)$ can be represented linearly by $P_i(l)$. Additionally, the local constraint projection operator $[\cdot]_{P_i^{min}}^{P_i^{max}}$ can be implemented by a total of four ReLU units as illustrated below as well as in Fig. 3.

$$[I]_{min}^{max} = min + \max(\max(A, 0) - min, 0) \quad (3)$$

where A is the projection operator $[\cdot]_0^{P_i^{max}}$ which can be implemented by two ReLU units:

$$A = [I]_0^{max} = max - \max(max - \max(I, 0), 0) \quad (4)$$

where I is the input.

Once $S_i(l)$ is obtained, we need to analyze implementation of $(n-1)P_i(l) + X_i(l)$, for convenience, which is denoted as $V_i(l)$:

$$V_i(l) = nP_i(l) - \frac{\sum_{i=1}^n P_i(l) - P_d}{n} \quad (5)$$

which shows that $V_i(l)$ can be represented linearly by $P_i(l)$. Finally we get $P_i(l+1)$ through a simple linear combination of $S_i(l)$ and $V_i(l)$. The whole implementation construction of HDO algorithm by using DNN is shown in Fig. 4.

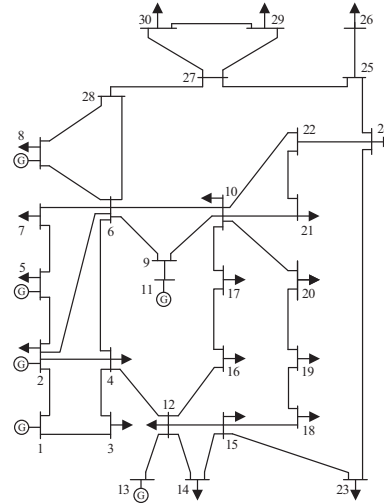


Figure 4: IEEE 30-bus system

From the above construction, it is clear that we need $O(l) = 6l$ layers with $O(nl) = 4nl$ neurons to approximate Algorithm 1 without error theoretically. ■

Remark 1 It is worthy pointing out that Theorem 1 gives the upper bound of the required number of hidden layers and their respective neurons. In practice, it is usually observed that much smaller networks are also able to achieve good approximation accuracy, which will be verified by the case studies in the next section.

4 Case Studies

4.1 IEEE 30-bus system

In this section, in order to test the effectiveness of proposed real-time OPA algorithm, the IEEE 30-bus system (shown in Fig. 5) is chosen as a test system. There are 6 dispatchable generators in this system, and the generator parameters are adopted from [8], which are also listed in Table 1. In addition, the nominal total load demand $P_d = \sum_{i=1}^{30} P_i^d = 283.4MW$.

4.2 Experiment setup

In this case study, the total load demand P_d is assumed to be randomly changed to simulate the load fluctuation as

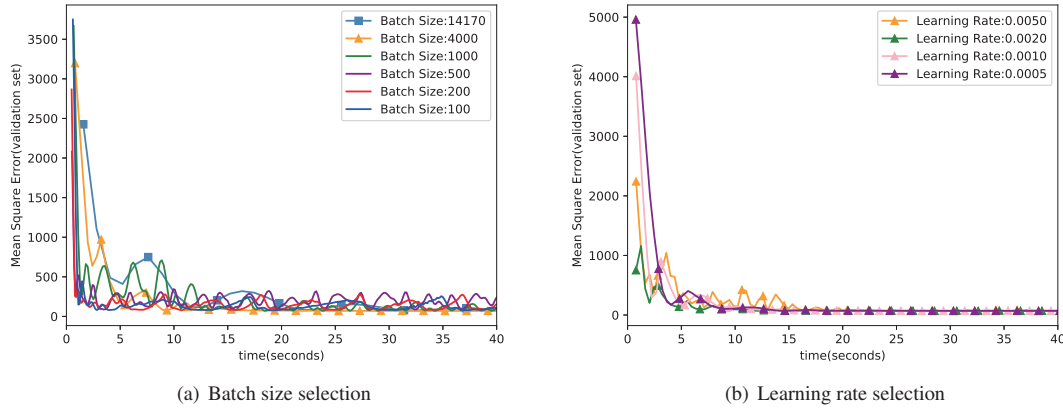


Figure 5: Parameter selection for IEEE-30 bus case

Table 1: Generator Parameters in IEEE 30-bus power system

DG	α_i [\$/MW^2h]	β_i [\$/MWh]	γ_i [\$/h]	$P_i^{\min}$ [MW]	$P_i^{\max}$ [MW]
1	0.00375	2	0	50	200
2	0.0175	1.75	0	20	80
5	0.0625	1.0	0	15	50
8	0.00834	3.25	0	10	35
11	0.025	3.0	0	10	30
13	0.025	3.0	0	12	40

well as the renewable sources penetration. Specifically, P_d varies in the range of [141.7, 425.1], i.e., 50% variation from its nominal value. The data sampling resolution is set as 0.01 in this case. In this experiment, the odd array is used for training, and the even array is used for testing. In this way, we get 14170 load cases as the input for the training data set. Meanwhile, by feeding these load cases into the HDO algorithm, the corresponding optimal power outputs can be obtained, which can be used as the output for the training data set.

The proposed DNN-based approach is implemented in python 3.7.2 with TensorFlow 1.0.0 on a server computer with two 10-core Intel Broadwell processors, two Nvidia K40 Graphical Processing Units (GPUs), and 128GB of memory. The GPUs are used in the training stage to reduce the training time, but are not used in the testing stage. For all our numerical results, we use a network with one input layer, one output layer, and three hidden layers with each hidden layer containing 200 neurons.

To find parameters for the training algorithm, we perform cross-validation for this case study. The impact of the batch size and learning rate on the MSE evaluated on the validation data set early in training is studied. As observed in Fig. 6 (a), larger batch size leads to slower convergence, while smaller batch size incurs unstable convergence behavior. In addition, it is also observed from Fig. 6 (b) that the convergence property does not change too much among the selected learning rate between 0.0005 and 0.005. Taking the

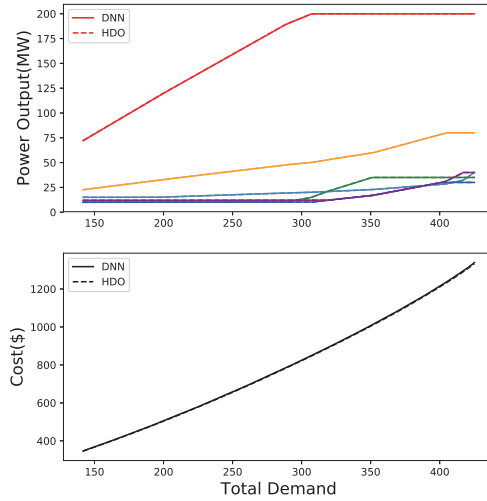


Figure 6: Comparison of the DNN-based approach and the HDO algorithm in [8]

above observations and practical concerns into consideration, we choose the batch size to be 1000 and the learning rate as 0.001.

4.3 On-line Optimal Power Allocation

Once the DNN is well trained, we compare the performance of our proposed DNN-based algorithm and the conventional iteration-based HDO algorithm in [8]. The results are shown in Figs. 7-8. As can be observed from Fig. 7, our DNN-based approach can achieve almost the same optimal results as the HDO algorithm. But it is worthy of highlighting that such similar performances are achieved at quite different computational costs. As shown in Fig. 8, the DNN-based approach only requires 0.63 ms, which is much shorter than 1.82 s in HDO algorithm, where it takes 1081407 iterations to converge the optimal point. It is foreseeable that our proposed DNN-based approach would

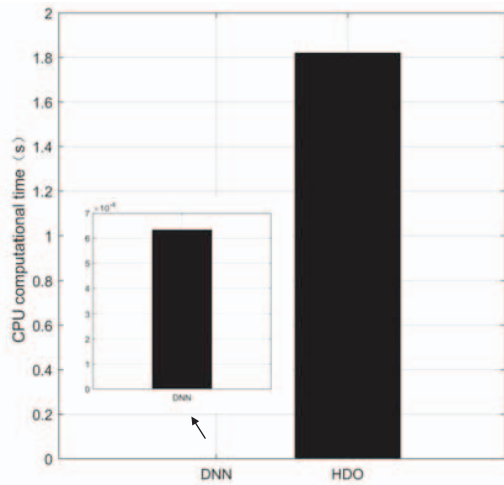


Figure 7: Computation time comparison between the proposed DNN-based approach and the HDO algorithm in [8]

have tremendous advantages over traditional numerical optimization algorithms when the system is becoming larger and larger. Hence it can be concluded our proposed DNN-based algorithm is more suitable for real-time OPA implementation in future smart grid systems.

5 Conclusion

A new learning-based on-line optimization algorithm is presented in this paper to allocate power among all the dispatchable DERs in smart grid systems. It is theoretically analyzed that the proposed deep neural network can well approximate the existing OPA optimization algorithm (e.g., HDO algorithm in [8]) by only using finite hidden layers and neurons. Besides, the experimental results show that the proposed DNN-based learning approach can achieve similar optimal results as the conventional HDO algorithm but with much lower computational time. These results indicate that our proposed DNN-based method is more suitable for real-time optimal power allocation in smart grid systems, especially when the system scales up.

It is noted that the current work employs a single DNN to approximate the whole HDO algorithm. How to approximate the HDO algorithm in a distributed way, i.e., using distributed DNNs, may be considered as future work. In addition, it is also an exciting but challenging work to use DNN to solve some non-convex OPA problems in the future.

REFERENCES

- [1] X. Yu, Y. Xue, "Smart grids: A cyber-physical systems perspective," *Proceedings of the IEEE*, vol. 104, no. 5, pp. 1058 - 1070, May 2016.
- [2] F. Guo, L. Wang, C. Wen, D. Zhang, Q. Xu, "Distributed voltage restoration and current sharing control in islanded DC microgrid systems without continuous communication," *IEEE Transactions on Industrial Electronics*, vol. 67, no. 4, pp. 3043-3053, April, 2020.
- [3] G. Chen, F. L. Lewis, E. N. Feng and Y.-D. Song, "Distributed optimal active power control of multiple generation systems," *IEEE Trans. Ind. Elect.*, vol. 62, no. 11, pp. 7079 - 7090, Nov. 2016.
- [4] A. Askarzadeh, "A memory-based genetic algorithm for optimization of power generation in a microgrid," *IEEE Trans. Sust. Ener.*, vol. 9, no. 3, pp. 1081 - 1089, July 2018.
- [5] F. Guo, C. Wen, J. Mao, and Y.-D. Song, "Distributed economic dispatch for smart grids with random wind power," *IEEE Trans. Smart Grid*, vol. 7, no. 3, pp. 1572 - 1089, May 2016.
- [6] Z.-L. Gaing, "Particle swarm optimization to solving the economic dispatch considering the generator constraints," *IEEE Trans. Power Syst.*, vol. 18, no. 3, pp. 1187 - 1195, Aug. 2003.
- [7] G. Chen, Q. Yang, "An ADMM-based distributed algorithm for economic dispatch in islanded microgrids," *IEEE Trans. Ind. Infor.*, vol. 14, no. 9, pp. 3892-3903, Sept. 2018.
- [8] F. Guo, C. Wen, J. Mao, J. Chen, and Y.-D. Song, "Hierarchical decentralized optimization architecture for economic dispatch: A new approach for large-scale power system," *IEEE Trans. Ind. Infor.*, vol. 14, no. 2, pp. 523 - 534, Feb., 2018.
- [9] Y. Zheng, Y. Song, D. J. Hill, and Y. Zhang, "Multiagent system based microgrid energy management via asynchronous consensus ADMM," *IEEE Trans. Energy Conversion*, vol. 33, no. 2, pp. 886-888, 2018.
- [10] H. Sun, X. Chen, Q. Shi, M. Hong, X. Fu, and N. D. Sidiropoulos, "Learning to optimize: Training deep neural networks for interface management," *IEEE Trans. Sig. Proc.*, vol. 66, no. 20, pp. 5438 - 5453, Oct., 2018.
- [11] M. Eisen, C. Zhang, L. Chamon, D. Lee, and A. Ribeiro, "Learning optimal resource allocations in wireless systems," *IEEE Trans. Sig. Proc.*, vol. 67, no. 10, pp. 2775 - 2790, May, 2019.
- [12] K. Li, J. Malik, "Learning to optimize," published online, arXiv: 1606.01885v1, 2016.
- [13] M. Eisen, C. Zhang, L. Chamon, D. Lee, and A. Ribeiro, "Learning optimal resource allocations in wireless systems," *IEEE Trans. Sig. Proc.*, vol. 67, no. 10, pp. 2775 - 2790, May, 2019.
- [14] J. Nanda, A. Sachan, L. Pradhan, L. L. Lai, M. Prasad, "Application of artificial neural network to economic load dispatch," in *Proc. of the 4th Intern. Confer. Advan. in Power Syst. Contr., Operat. and Manag.*, pp. 707 - 711, Hong Kong, 1997.
- [15] S. Panta, S. Premrudeepreechacharn, "Economic dispatch for power generation using artificial neural network," in *Proc. of the 7th Intern. Confer. on Power Electr.*, pp. 558 - 562, Daegu, Korea, 2007.
- [16] S. Liang and R. Srikant, "Why deep neural networks for function approximation?" in *Proc. of Intern. Confer. on Learn. Represnet.*, 2017.
- [17] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359-366, 1989.
- [18] X. Le, S. Chen, Z. Yan, and J. Xi, "A neurodynamic approach to distributed optimization with globally coupled constraints," *IEEE Trans. Cybern.*, vol. 48, no. 11, pp. 3149 - 3158, Nov., 2018.
- [19] F. Liang, C. Shen, W. Yu, and F. Wu, "Towards optimal power control via ensembling deep neural networks," published online, <https://arxiv.org/abs/1807.10025>.
- [20] Kingma D P, Ba J. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, 2014.